Numerical Simulation of the Gray-Scott Reaction-Diffusion Model via GPU Acceleration

April 1, 2026

Abstract

This document establishes the theoretical and numerical foundations required for simulating the Gray-Scott reaction-diffusion system using high-performance parallel computing. We begin by deriving the continuous partial differential equations (PDEs) from fundamental chemical kinetics and the physical principles of diffusion. Subsequently, we construct discrete numerical approximations using the Finite Difference Method (FDM) for spatial operators and Explicit Euler integration for temporal evolution. Finally, we map these discrete mathematical constructs onto the massively parallel architecture of a Graphics Processing Unit (GPU) using Compute Unified Device Architecture (CUDA), providing a rigorous blueprint for a high-performance C++ solver.

Contents

1 Introduction to Reaction-Diffusion Systems **3**
1.1 The Phenomenon of Diffusion 3
1.2 The Addition of Kinetics (Reaction) 4

2 The Gray-Scott Kinetic Model **4**
2.1 Chemical Reactions and the Open System Model 5
2.2 Deriving the Gray-Scott PDEs 5

3 Spatial Discretization: The Finite Difference Method **6**
3.1 Domain Definition and the Computational Grid 6
3.2 Taylor Series Derivation of the Laplacian 6
3.3 Truncation Error Analysis 7
3.4 The 5-Point Stencil 7
3.5 Periodic Boundary Conditions 8

4 Temporal Discretization: Explicit Euler Integration **8**
4.1 The Discrete Time Domain 8
4.2 Derivation of the Forward Euler Method 9
4.3 The Fully Discrete Gray-Scott Equations 9
4.4 Stability Analysis and the Numerical Speed Limit 10
4.5 Justification of Euler Integration for Dissipative Systems 11

5 Parallel Mapping and CUDA Architecture **11**
5.1 Linearizing the Spatial Domain 11
5.2 Scalable Grid Decomposition and the Stride Access Pattern 11

1 Introduction to Reaction-Diffusion Systems

Before diving into the specific equations of the Gray-Scott model, we must first understand the general class of physical systems it belongs to: reaction-diffusion systems. These systems describe how the concentrations of one or more substances distributed in space change under the influence of two simultaneous, often competing, processes: local chemical reactions and spatial diffusion.

1.1 The Phenomenon of Diffusion

Diffusion is a fundamental transport phenomenon where particles spread from regions of higher concentration to regions of lower concentration, driven microscopically by random thermal motion (Brownian motion).

Macroscopically, this transport is described by **Fick's First Law**, which postulates that the flow vector field $\mathbf{J}$ (often called the flux density, representing the flow of particles per unit area per unit time) is directly proportional to the negative spatial gradient of the concentration $C(\mathbf{x}, t)$:

$$\mathbf{J} = -D\nabla C$$

Here, D is the diffusion coefficient, and the negative sign indicates that the net flow occurs *down* the concentration gradient.

To understand how the concentration field evolves over time, we must enforce the **Principle of Mass Conservation**. Consider an arbitrary, fixed control volume V bounded by a closed surface S with an outward-pointing normal vector $\hat{\mathbf{n}}$.

We distinguish between the local flow vector field $\mathbf{J}$ and the macroscopic total flux. While $\mathbf{J}$ is a vector representing flow at a specific geometric point, integrating this field over a surface yields a scalar value known as the total flux. For our closed surface S, the surface integral $\oint_S \mathbf{J} \cdot \hat{\mathbf{n}}\, dS$ computes the total net rate at which mass crosses the boundary. Because the normal vector $\hat{\mathbf{n}}$ always points *outward*, the dot product $\mathbf{J} \cdot \hat{\mathbf{n}}$ is positive when mass flows out and negative when mass flows in. Therefore, the integral itself evaluates to the net *outward-going* mass. If the integral is negative, the net motion of mass is inward.

Assuming no chemical reactions are occurring yet (pure diffusion), the rate of change of the total mass accumulated by diffusion within the volume V must equal the net *inward* mass flow, which is just the negative of the flux integral:

$$\frac{d}{dt}\int_V C\, dV = -\oint_S \mathbf{J} \cdot \hat{\mathbf{n}}\, dS$$

Because the control volume V is fixed in space and time, we can bring the time derivative inside the integral as a partial derivative:

$$\int_V \frac{\partial C}{\partial t}\, dV = -\oint_S \mathbf{J} \cdot \hat{\mathbf{n}}\, dS$$

We now apply the **Divergence Theorem (Gauss's Theorem)** to convert the surface integral of the flow vector field into a volume integral of the divergence of that field:

$$\oint_S \mathbf{J} \cdot \hat{\mathbf{n}}\, dS = \int_V (\nabla \cdot \mathbf{J})\, dV$$

Substituting this back into our conservation equation yields:

$$\int_V \frac{\partial C}{\partial t}\, dV = -\int_V (\nabla \cdot \mathbf{J})\, dV$$

$$\int_V \left(\frac{\partial C}{\partial t} + \nabla \cdot \mathbf{J} \right) dV = 0$$

Because this integral must hold true for *any* arbitrary control volume V, the integrand itself must be identically zero everywhere in the domain. This gives us the differential form of the continuity equation:

$$\frac{\partial C}{\partial t} = -\nabla \cdot \mathbf{J}$$

Finally, we substitute Fick's First Law ($\mathbf{J} = -D\nabla C$) into this continuity equation to arrive at the diffusion equation (often called **Fick's Second Law**):

$$\frac{\partial C}{\partial t} = \nabla \cdot (D\nabla C)$$

For our specific simulation, we assume the diffusion coefficient D is a constant, uniform scalar across the entire spatial domain (isotropic and homogeneous diffusion). This allows us to pull D out of the divergence operator, yielding the final equation we will solve:

$$\frac{\partial C}{\partial t} = D\nabla^2 C$$

The term ∇^2 is the Laplace operator (or Laplacian, $\nabla \cdot \nabla$). Physically, the Laplacian measures the difference between the concentration at a specific point and the average concentration of its infinitesimal surroundings, dictating how the concentration profile naturally smooths out over time.

1.2 The Addition of Kinetics (Reaction)

While diffusion acts to smooth out concentration differences, chemical reactions create or destroy substances locally. If a substance C is created or consumed by a chemical reaction at a rate given by a function $R(C)$, the total rate of change of the concentration over time is simply the sum of the diffusion term and the reaction term.

For a system of N interacting chemical species represented by a concentration vector $\mathbf{u} = (u_1, u_2, \ldots, u_N)^T$, the general reaction-diffusion system is written as a coupled set of Partial Differential Equations (PDEs):

$$\frac{\partial \mathbf{u}}{\partial t} = \mathbf{D}\nabla^2 \mathbf{u} + \mathbf{R}(\mathbf{u})$$

Where $\mathbf{D}$ is a diagonal matrix of diffusion coefficients, and $\mathbf{R}(\mathbf{u})$ is a vector-valued function describing the local chemical kinetics. The interplay between the smoothing effect of $\mathbf{D}\nabla^2\mathbf{u}$ and the nonlinear dynamics of $\mathbf{R}(\mathbf{u})$ can give rise to complex, self-organizing spatial patterns, such as spots, stripes, and spirals.

2 The Gray-Scott Kinetic Model

The Gray-Scott model is a specific, well-studied two-component reaction-diffusion system. It simulates the interaction of two chemical species, conventionally labeled U and V.

2.1 Chemical Reactions and the Open System Model

The model is constructed on three fundamental processes occurring within an open, continuously fed system (analogous to a Continuously Stirred Tank Reactor, or CSTR):

1. **Autocatalysis:** $U + 2V \to 3V$. One molecule of U reacts with two molecules of V to produce three molecules of V.

2. **Decay:** $V \to P$. Species V decays into an inert, unmodeled product P.

3. **Continuous Flow (Feed and Drain):** Fresh fluid containing only species U is pumped into the system at a constant volumetric rate. To maintain a constant volume, the mixed fluid (containing both U and V) is continuously drained from the system at the exact same rate.

To convert the chemical interactions into mathematical rate equations, we utilize the **Law of Mass Action**, which dictates that the rate of a chemical reaction is directly proportional to the product of the concentrations of the reactants.

Let u and v be the spatial concentrations of species U and V. For the autocatalytic reaction ($U + 2V \to 3V$), the reaction rate is proportional to $u \cdot v \cdot v$, or uv^2. Because this reaction consumes one U and yields a net gain of one V, the kinetic rate of change is $-uv^2$ for species U and $+uv^2$ for species V.

2.2 Deriving the Gray-Scott PDEs

We now combine the spatial diffusion, the mass action kinetics, and the thermodynamic flow of the open system to construct the full governing equations.

Let F be the volumetric **feed rate** (flow rate) of the system. We assume the incoming fluid contains species U at a normalized background concentration of 1. Therefore, species U is continually added to the system at a constant rate of $F \cdot 1 = F$.

Simultaneously, the mixed fluid is drained at the same flow rate F. The removed fluid contains U at its current local concentration u, meaning U is removed at a rate of $F \cdot u$. The net rate of change for U due to the system's flow is the feed minus the drain: $F - Fu = F(1 - u)$.

Species V, however, is not present in the incoming feed. It is only subject to removal. It is flushed out by the bulk fluid drain at a rate of $F \cdot v$. Additionally, it undergoes the independent chemical decay into an inert product at a specific rate k, meaning it is chemically lost at a rate of $k \cdot v$. The total removal rate for V is therefore $-Fv - kv = -(F + k)v$.

Assembling all parts—diffusion, autocatalysis, specific decay, and continuous flow—we arrive at the classic dimensionless formulation of the Gray-Scott PDEs:

$$\frac{\partial u}{\partial t} = D_U \nabla^2 u - uv^2 + F(1 - u)$$
$$\frac{\partial v}{\partial t} = D_V \nabla^2 v + uv^2 - (F + k)v$$

Term-by-term breakdown:

- $\frac{\partial u}{\partial t}, \frac{\partial v}{\partial t}$: The local rate of change of the concentrations of U and V over time.

- $D_U \nabla^2 u, D_V \nabla^2 v$: Spatial diffusion. D_U is typically much larger than D_V, meaning U spreads much faster than V. This differential diffusion is the primary mechanism driving pattern formation.
- $-uv^2$ (in U) and $+uv^2$ (in V): The non-linear autocatalytic reaction.
- $F(1-u)$: The net flow of U, representing the constant influx of fresh U ($+F$) minus the outflux of mixed U ($-Fu$).
- $-(F+k)v$: The total removal of V, representing the physical outflux of the mixed fluid ($-Fv$) combined with the chemical decay ($-kv$).

These continuous PDEs represent the exact physical model. However, digital computers cannot analytically solve these continuous functions for arbitrary domains. To implement our CUDA solver, we must discretize these continuous space and time dimensions into finite grids and steps.

3 Spatial Discretization: The Finite Difference Method

To simulate the continuous PDEs computationally, we must restrict our evaluation to a finite set of points in space. This process is known as discretization. We will employ the Finite Difference Method (FDM), which approximates continuous derivative operators (like the Laplacian, ∇^2) using algebraic combinations of values at discrete grid points.

3.1 Domain Definition and the Computational Grid

Let our continuous spatial domain be a two-dimensional rectangular region $\Omega = [0, L_x] \times [0, L_y]$. We map this continuous domain onto a discrete computational grid Ω_h consisting of $N_x \times N_y$ uniformly spaced nodes.

The physical distances between adjacent nodes in the x and y directions are the spatial step sizes, denoted as Δx and Δy, respectively:

$$\Delta x = \frac{L_x}{N_x - 1}, \quad \Delta y = \frac{L_y}{N_y - 1}$$

A specific spatial coordinate (x, y) is replaced by a discrete index pair (i, j), such that $x_i = i\Delta x$ and $y_j = j\Delta y$, where $i \in \{0, 1, \ldots, N_x - 1\}$ and $j \in \{0, 1, \ldots, N_y - 1\}$.

Consequently, we must clearly distinguish the exact continuous concentration function $u(x, y, t)$ from our discrete numerical approximation. We define $u_{i,j}$ as the approximated concentration at grid node (x_i, y_j):

$$u_{i,j}(t) \approx u(x_i, y_j, t)$$

3.2 Taylor Series Derivation of the Laplacian

The most challenging spatial term to discretize in the Gray-Scott PDEs is the Laplace operator, $\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$. To derive a discrete algebraic approximation for these second spatial derivatives, we rely on the Taylor series expansion.

Consider a fixed point (x_i, y_j) and a fixed time t. For brevity, we will write the continuous function evaluated along the x-axis as $u(x)$. We expand the continuous function at the adjacent grid points, $u(x_i + \Delta x)$ and $u(x_i - \Delta x)$, around the central point x_i:

$$u(x_i + \Delta x) = u(x_i) + \Delta x \frac{\partial u}{\partial x} + \frac{\Delta x^2}{2!}\frac{\partial^2 u}{\partial x^2} + \frac{\Delta x^3}{3!}\frac{\partial^3 u}{\partial x^3} + \frac{\Delta x^4}{4!}\frac{\partial^4 u}{\partial x^4} + \mathcal{O}(\Delta x^5)$$

$$u(x_i - \Delta x) = u(x_i) - \Delta x \frac{\partial u}{\partial x} + \frac{\Delta x^2}{2!}\frac{\partial^2 u}{\partial x^2} - \frac{\Delta x^3}{3!}\frac{\partial^3 u}{\partial x^3} + \frac{\Delta x^4}{4!}\frac{\partial^4 u}{\partial x^4} - \mathcal{O}(\Delta x^5)$$

By adding these two equations together, the odd-derivative terms (the first and third derivatives) perfectly cancel out, yielding:

$$u(x_i + \Delta x) + u(x_i - \Delta x) = 2u(x_i) + \Delta x^2 \frac{\partial^2 u}{\partial x^2} + 2\frac{\Delta x^4}{4!}\frac{\partial^4 u}{\partial x^4} + \mathcal{O}(\Delta x^6)$$

We now algebraically isolate the second derivative, $\frac{\partial^2 u}{\partial x^2}$, which is the term we wish to approximate:

$$\frac{\partial^2 u}{\partial x^2} = \frac{u(x_i + \Delta x) - 2u(x_i) + u(x_i - \Delta x)}{\Delta x^2} - \frac{\Delta x^2}{12}\frac{\partial^4 u}{\partial x^4} - \mathcal{O}(\Delta x^4)$$

Switching back to our discrete notation ($u_{i,j} \approx u(x_i, y_j)$), we obtain the **Central Difference Approximation** for the second spatial derivative in the x-direction:

$$\left(\frac{\partial^2 u}{\partial x^2}\right)_{i,j} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}$$

3.3 Truncation Error Analysis

The mathematical derivation above explicitly reveals the accuracy of our approximation. The terms we discarded to form the discrete algebraic equation constitute the **truncation error**. The leading error term is $-\frac{\Delta x^2}{12}\frac{\partial^4 u}{\partial x^4}$.

Because the largest discarded term scales with the square of the grid spacing (Δx^2), we formally classify this discrete approximation as **second-order accurate in space**, denoted as $\mathcal{O}(\Delta x^2)$. This rigor ensures us that if we halve the distance between our grid nodes, the error associated with spatial discretization will decrease by a factor of four.

3.4 The 5-Point Stencil

Applying the exact same Taylor series methodology to the y-direction yields an analogous discrete approximation:

$$\left(\frac{\partial^2 u}{\partial y^2}\right)_{i,j} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}$$

The full continuous Laplacian ($\nabla^2 u$) is simply the sum of these two perpendicular second derivatives. For standard numerical simulations of the Gray-Scott model, it is convention to utilize a uniform square grid where the horizontal and vertical spacings are identical ($\Delta x = \Delta y = h$). Under this geometric assumption, the denominators unify, allowing us to combine the numerators:

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2}$$

This formula represents the discrete **5-Point Stencil**. To evaluate the Laplacian at any central node (i, j), the algorithm queries the concentration at the node itself (multiplying it by -4) and adds the concentrations of its four immediate orthogonal neighbors (North, South, East, and West). This exact algebraic operation is what we will later map to our parallel CUDA kernels.

3.5 Periodic Boundary Conditions

A final structural requirement for the discrete grid is the mathematical treatment of nodes residing exactly on the physical boundaries (e.g., $i = 0$ or $i = N_x - 1$). For these boundary nodes, the 5-point stencil requires querying a neighbor that lies outside the computational domain, resulting in an undefined index.

To resolve this without introducing artificial barrier effects, we impose **Periodic Boundary Conditions**. This mathematically wraps the geometry of the flat rectangular grid into a toroidal topology (a donut shape). A node on the rightmost edge treats the leftmost edge as its immediate right-hand neighbor, and vice versa.

Mathematically, this is defined by enforcing the congruency:

$$u_{-1,j} \equiv u_{N_x-1,j} \quad \text{and} \quad u_{N_x,j} \equiv u_{0,j}$$

$$u_{i,-1} \equiv u_{i,N_y-1} \quad \text{and} \quad u_{i,N_y} \equiv u_{i,0}$$

In our C++ implementation, this will be gracefully handled using integer modulo arithmetic when calculating neighbor indices, ensuring that the simulation acts as an infinitely repeating continuous domain without physical walls.

4 Temporal Discretization: Explicit Euler Integration

With the continuous space Ω mapped to a discrete grid Ω_h, we must similarly constrain the continuous time t into a sequence of discrete temporal steps. This transforms our system of Partial Differential Equations into a system of algebraic update rules that a processor can iteratively execute.

4.1 The Discrete Time Domain

We advance the simulation through time in uniform increments, denoted as the time step Δt. The continuous time variable t is replaced by discrete intervals $t^n = n\Delta t$, where the integer $n \geq 0$ represents the current temporal iteration.

To seamlessly integrate time and space in our mathematical notation, we append n as a superscript to our discrete concentration variables. Therefore, $u^n_{i,j}$ rigorously defines the computed concentration of species U at spatial grid coordinate (i, j) exactly at time step n:

$$u^n_{i,j} \approx u(x_i, y_j, t^n)$$

Our computational goal is an Initial Value Problem (IVP): given the complete spatial state of the system at the current time step n, we must compute the completely new spatial state at the future time step $n + 1$.

4.2 Derivation of the Forward Euler Method

To achieve this, we return to the Taylor series expansion, this time applying it exclusively to the temporal dimension. We expand the continuous function $u(t)$ into the future by one step Δt, evaluating it around the current time t^n:

$$u(t^{n+1}) = u(t^n + \Delta t) = u(t^n) + \Delta t \left.\frac{\partial u}{\partial t}\right|_{t^n} + \frac{\Delta t^2}{2!} \left.\frac{\partial^2 u}{\partial t^2}\right|_{t^n} + \mathcal{O}(\Delta t^3)$$

By truncating this series after the first derivative, we algebraically isolate the rate of change:

$$\left.\frac{\partial u}{\partial t}\right|_{t^n} = \frac{u(t^{n+1}) - u(t^n)}{\Delta t} - \frac{\Delta t}{2}\frac{\partial^2 u}{\partial t^2} - \mathcal{O}(\Delta t^2)$$

Substituting our discrete notation, we obtain the **Forward Difference Approximation** for the temporal derivative:

$$\left(\frac{\partial u}{\partial t}\right)^n_{i,j} \approx \frac{u^{n+1}_{i,j} - u^n_{i,j}}{\Delta t}$$

Because the leading truncated error term scales linearly with Δt, this temporal discretization is classified as **first-order accurate in time**, denoted as $\mathcal{O}(\Delta t)$.

4.3 The Fully Discrete Gray-Scott Equations

We are now prepared to build the final update algorithms. We take the continuous Gray-Scott PDEs (derived in Section 2) and substitute our spatial 5-point stencil (Section 3) and our temporal forward difference (Section 4.2).

For species U, the substitution yields:

$$\frac{u^{n+1}_{i,j} - u^n_{i,j}}{\Delta t} = D_U \left(\frac{u^n_{i+1,j} + u^n_{i-1,j} + u^n_{i,j+1} + u^n_{i,j-1} - 4u^n_{i,j}}{h^2} \right) - u^n_{i,j}(v^n_{i,j})^2 + F(1 - u^n_{i,j})$$

By multiplying by Δt and moving $u^n_{i,j}$ to the right-hand side, we isolate the unknown future state $u^{n+1}_{i,j}$. Applying this algebraic rearrangement to both species results in the **Explicit Euler Update Equations**:

$$\begin{aligned} u^{n+1}_{i,j} &= u^n_{i,j} + \Delta t \left[D_U \nabla^2_{discrete} u^n_{i,j} - u^n_{i,j}(v^n_{i,j})^2 + F(1 - u^n_{i,j}) \right] \\ v^{n+1}_{i,j} &= v^n_{i,j} + \Delta t \left[D_V \nabla^2_{discrete} v^n_{i,j} + u^n_{i,j}(v^n_{i,j})^2 - (F + k) v^n_{i,j} \right] \end{aligned}$$

(Where $\nabla^2_{discrete}$ represents the 5-point spatial stencil calculated entirely at time n.)

This formulation is termed **explicit** because the future state at node (i, j) depends *only* on information already known from the current time step n. This property is exactly why we select Explicit Euler for GPU computing: every spatial node can compute its future state entirely independently of its neighbors' future states, allowing for massive, lock-step parallelization across thousands of CUDA threads.

4.4 Stability Analysis and the Numerical Speed Limit

While the explicit method provides massive parallelization advantages, it introduces a critical mathematical vulnerability: it is **conditionally stable**. To understand why explicit methods can fail, we must analyze the physical flow of mass represented by our discrete equations.

Let us isolate the pure diffusion term from our U equation (temporarily ignoring the reaction and flow kinetics, $R(u, v)$) and explicitly write out the 5-point stencil:

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \frac{D_U \Delta t}{h^2} \left(u_{i+1,j}^{n} + u_{i-1,j}^{n} + u_{i,j+1}^{n} + u_{i,j-1}^{n} - 4u_{i,j}^{n} \right) + \Delta t R(u^n, v^n)$$

By algebraically regrouping the terms associated with the central cell $u_{i,j}^{n}$, we reveal a more physically intuitive form of the update equation:

$$u_{i,j}^{n+1} = \left(1 - \frac{4D_U \Delta t}{h^2} \right) u_{i,j}^{n} + \frac{D_U \Delta t}{h^2} \left(u_{i+1,j}^{n} + u_{i-1,j}^{n} + u_{i,j+1}^{n} + u_{i,j-1}^{n} \right) + \Delta t R(u^n, v^n)$$

This equation clearly states that the future concentration $u_{i,j}^{n+1}$ is a combination of mass arriving from the four neighbors, plus a fraction of the mass that originally resided in the central cell itself. The coefficient $\left(1 - \frac{4D_U \Delta t}{h^2}\right)$ dictates exactly what fraction of original mass remains after outward diffusion.

Because concentration represents physical mass, it cannot mathematically drop below zero. Therefore, this coefficient *must* remain positive. If we select a time step Δt that is too large, the term $\frac{4D_U \Delta t}{h^2}$ will exceed 1, rendering the coefficient negative.

Physically, a negative coefficient means the numerical method is attempting to extract more mass from the discrete control volume than it currently possesses. Once a single cell dips into a negative concentration, the neighboring cells perceive a massive numerical void and rapidly diffuse mass into it on the next time step, overcompensating and creating an artificial spike. This triggers a catastrophic chain reaction of wildly oscillating, unphysical concentrations—the hallmark of numerical instability.

To prevent this violation of mass positivity, we must strictly bound the time step such that the coefficient remains non-negative:

$$1 - \frac{4D_U \Delta t}{h^2} \geq 0 \implies \Delta t \leq \frac{h^2}{4D_U}$$

This boundary represents a **numerical speed limit**. Our explicit 5-point stencil has a strict domain of dependence: it only considers data from exactly one adjacent cell away. Therefore, numerical information can only propagate at a maximum speed of one grid cell per Δt. If we configure Δt such that the physical diffusion rate attempts to move mass further than one grid cell in a single step, we violate the solver's domain of dependence, resulting in failure.

Because the Gray-Scott system involves two diffusing species, the stability limit is governed strictly by the species that diffuses the fastest. Thus, our foundational condition for a stable simulation is:

$$\Delta t \leq \frac{h^2}{4 \max(D_U, D_V)}$$

4.5 Justification of Euler Integration for Dissipative Systems

It is a well-known axiom in computational physics—particularly within molecular dynamics and orbital mechanics—that the Forward Euler method should generally be avoided. In those fields, simulations typically model *conservative* Hamiltonian systems where total energy and momentum must remain constant. Explicit Euler is a non-symplectic integrator; its first-order truncation error acts mathematically as artificial positive friction, injecting spurious numerical energy into the system and causing orbits to continually expand and fail.

However, this axiom does not apply to the Gray-Scott model because the latter is fundamentally a *dissipative* system. Governed by parabolic partial differential equations, the system is strictly open: it is driven by continuous feed and drain rates, and spatial variations are constantly smoothed out by the diffusion operator. There is no conserved Hamiltonian "energy" to protect.

While Explicit Euler still introduces a temporal truncation error of $\mathcal{O}(\Delta t)$, any artificial numerical amplification is naturally suppressed by the physical dissipation due to the continuous diffusion, provided the stability criterion ($\Delta t \leq \frac{h^2}{4D}$) is respected. In exchange for this tolerable loss in absolute temporal accuracy, the explicit single-step nature of Euler requires significantly fewer memory read/write operations compared to higher-order multi-step methods (like Runge-Kutta). Because finite difference PDE solvers on GPUs are almost entirely memory-bound rather than compute-bound, this minimization of memory traffic provides immense parallel performance benefits.

5 Parallel Mapping and CUDA Architecture

To translate our discretized mathematical framework into a high-performance C++ solver, we must map our abstract spatial grid to the physical execution model of an NVIDIA Graphics Processing Unit (GPU) using the Compute Unified Device Architecture (CUDA).

5.1 Linearizing the Spatial Domain

Our mathematical derivations assumed a two-dimensional domain with concentration matrices $u_{i,j}$ and $v_{i,j}$. However, GPU memory (VRAM) is physically linear. Therefore, we must define a bijective mapping function to flatten our 2D spatial coordinates (i, j) into a 1D memory index k.

We will employ standard **row-major ordering**. For a grid of dimensions N_x (width) and N_y (height), the 1D index k for a cell at row j and column i is calculated as:

$$k = j \cdot N_x + i$$

This mapping guarantees that adjacent cells in the same row (i and $i+1$) are stored contiguously in physical VRAM. This is a critical architectural requirement for CUDA performance, as it allows sequential threads to achieve *memory coalescing*—pulling chunks of adjacent data from VRAM in a single, highly efficient transaction.

5.2 Scalable Grid Decomposition and the Stride Access Pattern

To execute the integration over our domain, we launch a CUDA computational grid partitioned into Blocks, with each Block containing a grid of Threads. A naive implementation

maps exactly one Thread to one discrete cell (i, j). However, this approach tightly couples the simulation size to the maximum concurrent threads permitted by the hardware limits.

To achieve architectural decoupling, we employ the **2D Grid-Stride Loop** pattern. In this paradigm, we treat the hardware grid not as a 1-to-1 map of the simulation, but as a flexible parallel processing pool that "strides" over the simulation domain.

Let G_x and G_y represent the total number of hardware threads spanned across the x and y dimensions of the launched CUDA grid:

$$G_x = \text{gridDim.x} \cdot \text{blockDim.x}$$

$$G_y = \text{gridDim.y} \cdot \text{blockDim.y}$$

Each thread determines its initial starting coordinate (i_{start}, j_{start}) based on its unique hardware identification:

$$i_{start} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$$

$$j_{start} = \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y}$$

Once a thread computes the spatial update for its initial target cell $k = j \cdot N_x + i$, it does not terminate. Instead, it enters a nested loop, geometrically jumping forward by the total grid span (G_x horizontally, G_y vertically) to compute the next available target cell. It repeats this striding cycle until it traverses past the physical boundaries of the simulation domain (N_x, N_y).

This technique fundamentally decouples the software geometry from the hardware topology. A small computational grid can effortlessly traverse a massive simulation domain over multiple strides, maximizing hardware utilization (occupancy) while retaining the critical contiguous memory access (coalescing) defined by our row-major data layout.